

DinusNexus

Unified AI Digital Campus Worker

Dokumentasi Produk, Arsitektur, Workflow, dan Rencana Development

Documentation Book

Disusun 9 Oktober 2026

Baseline perencanaan — fitur berstatus rencana sampai dibuktikan oleh kode dan pengujian.

Daftar Isi

1. [README — Gambaran Produk](#)
2. [AGENTS — Instruksi Coding Agent](#)
3. [CONTRIBUTING — Git Workflow](#)
4. [Product Scope](#)
5. [Architecture Baseline](#)
6. [Stateful Workflows](#)
7. [Development Plan & Definition of Done](#)
8. [Rencana Development Paralel Tim](#)
9. [UI/UX Rules](#)
10. [Data, Security & Human Oversight](#)
11. [AI Token Efficiency & Measurement](#)
12. [Azure Deployment Baseline](#)
13. [Decision Register](#)
14. [Markdown Documentation Policy](#)

Dokumen ini menggabungkan seluruh Markdown proyek dalam urutan yang mendukung pemahaman: konteks → aturan kerja → scope → arsitektur → workflow → delivery → keamanan → deployment → keputusan.

README — Gambaran Produk

DinusNexus

Unified AI Digital Campus Worker — satu workspace untuk membantu berbagai peran operasional kampus menjalankan pekerjaan berbasis workflow, sumber data, dokumen, dan AI.

Status dokumen: baseline produk dan aturan kolaborasi yang sudah disepakati.
Detail yang belum diputuskan secara eksplisit tetap ditandai `TBD`; jangan menganggap usulan sebagai keputusan final.

Visi

DinusNexus menyatukan campus workers dalam satu platform percakapan dan operasional. Pengguna dapat mengirim instruksi, mengunggah dokumen, melihat proses eksekusi, memeriksa bukti dan hasil, serta tetap memegang kendali atas keputusan yang berdampak penting.

Campus roles

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

Scope prototype

- Semua delapan role menjadi bagian dari satu workspace dan arsitektur bersama.
- **IT Helpdesk adalah workflow pertama yang harus selesai secara end-to-end.**
- Campus Twin dimasukkan sebagai kemampuan tambahan yang memberi konteks lokasi/infrastruktur pada workflow terkait. Prioritas visual awal adalah peta/visualisasi 2D interaktif; 3D bukan syarat MVP.
- MVP menggunakan pola stateful workflow agent: pekerjaan memiliki status, langkah eksekusi, evidence/sumber, hasil, dan riwayat.
- UI harus mendukung chat, upload dokumen, input baru saat demo, tampilan proses yang benar-benar berasal dari backend, sumber/bukti, hasil, dan history yang bertahan setelah refresh.
- Manusia tetap memegang kendali atas tindakan sensitif melalui approval/otorisasi.
- Target deployment: Azure, memanfaatkan kredit Azure yang tersedia sekitar US\$100. Layanan dan konfigurasi Azure final masih TBD.
- OpenCode digunakan untuk membantu coding. API key/token disediakan panitia; model, endpoint, format autentikasi, dan batasannya masih TBD.

Kriteria implementasi minimum

Satu workflow operasional harus berjalan dari input sampai output melalui UI yang dapat dipakai; minimal ada satu sumber data atau tool di luar LLM; status/sumber/ringkasan dan history harus terlihat; input baru harus bisa dicoba saat live demo; data simulasi harus diberi label; setup dan hasil pengujian harus didokumentasikan.

Dokumen proyek

- [AGENTS.md](#): instruksi untuk coding agent/OpenCode.
- [CONTRIBUTING.md](#): branching, commit, dan pull request.
- [docs/PRODUCT_SCOPE.md](#): produk, delapan role, batas MVP.
- [docs/ARCHITECTURE.md](#): arsitektur konseptual dan tanggung jawab komponen.
- [docs/UI_UX.md](#): aturan UX dan tampilan.

- [docs/WORKFLOWS.md](#): alur stateful workflow dan MVP Helpdesk.
- [docs/DATA_SECURITY.md](#): sumber data, dokumen, keamanan, dan human approval.
- [docs/TOKEN EFFICIENCY.md](#): pencatatan dan optimasi token.
- [docs/AZURE_DEPLOYMENT.md](#): prinsip deployment Azure dan batas biaya.
- [docs/DOCUMENTATION_POLICY.md](#): aturan file Markdown dan pemeliharannya.
- [docs/DEVELOPMENT_PLAN.md](#): urutan implementasi dan Definition of Done.
- [docs/TEAM_DEVELOPMENT_PLAN.md](#): pembagian tiga role, kerja paralel, checkpoint, dan workflow GitHub.
- [docs/DECISIONS.md](#): keputusan final versus hal yang belum ditentukan.

Cara menggunakan dokumen ini

- 1.
- 2.
- 3.
- 4.

Catatan integritas

Dokumen ini adalah baseline perencanaan, bukan klaim bahwa fitur sudah diimplementasikan. Semua fitur baru berstatus rencana sampai kode dan test membuktikannya. Cantumkan penggunaan OpenCode, model/API panitia, library, template, dan kontribusi tim pada disclosure submission sesuai aturan hackathon.

AGENTS — Instruksi Coding Agent

AGENTS.md — Instruksi Coding Agent DinusNexus

Instruksi ini berlaku untuk OpenCode dan coding agent lain yang digunakan dalam repository.

Tujuan

Implementasikan DinusNexus sebagai Unified AI Digital Campus Worker. Bangun fondasi yang dapat mendukung delapan campus role dalam satu workspace, tetapi prioritaskan workflow IT Helpdesk yang benar-benar berjalan dari input sampai output. Campus Twin 2D interaktif adalah fitur pendukung, bukan pengganti workflow operasional.

Aturan wajib

- 1.
- 2.
- 3.
- 4.

- 5.
- 6.
- 7.
- 8.
- 9.
- 10.
- 11.
- 12.

Urutan kerja agent

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Prioritas produk

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

Coding style

- Ikuti formatter, konvensi penamaan, pola folder, dan library yang sudah dipakai di repo.
- Hindari duplikasi logika lintas role; gunakan service/tool bersama jika tepat.
- Validasi input di server, bukan hanya frontend.
- Tangani timeout, error provider, input kosong/tidak valid, dan kegagalan tool secara eksplisit.
- Gunakan tipe/schema terstruktur untuk kontrak API serta hasil workflow jika stack mendukungnya.
- Jangan menambahkan dependency baru tanpa kebutuhan jelas; dokumentasikan alasan jika dependency signifikan ditambahkan.

Batas otoritas

Coding agent boleh mengimplementasikan fitur sesuai keputusan terdokumentasi. Coding agent tidak boleh mengubah keputusan produk final, membuka data sensitif, melakukan deployment berbiaya, mengubah resource cloud, atau mengirim/mengubah data eksternal tanpa persetujuan manusia.

CONTRIBUTING — Git Workflow

Contributing & Git Workflow

Dokumen ini menetapkan aturan kolaborasi Git DinusNexus untuk menjaga `main` selalu stabil dan memudahkan pekerjaan paralel.

Branch utama

- `main`: branch stabil untuk integrasi yang sudah diperiksa dan kandidat demo/deployment.
- Semua pekerjaan dilakukan di branch pendek yang dibuat dari `main` terbaru.
- Jangan commit langsung ke `main`, kecuali tim secara eksplisit menyepakati pengecualian darurat.

Format nama branch

Gunakan huruf kecil, tanda hubung, dan prefix berikut:

- `feat/<ringkasan>` — fitur baru, misalnya `feat/it-helpdesk-workflow`
- `fix/<ringkasan>` — perbaikan bug, misalnya `fix/task-status-refresh`
- `docs/<ringkasan>` — dokumentasi, misalnya `docs/architecture-baseline`
- `refactor/<ringkasan>` — restrukturisasi tanpa mengubah perilaku yang dimaksud
- `test/<ringkasan>` — penambahan/perbaikan test
- `chore/<ringkasan>` — tooling, konfigurasi, dependency, atau pemeliharaan
- `design/<ringkasan>` — design system/UI polish bila perubahan tidak cocok sebagai fitur

Jaga satu branch untuk satu tujuan yang dapat dijelaskan. Hindari branch seperti `update`, `fix-now`, atau nama orang tanpa konteks.

Alur kerja

1.

```
git status --short
```

1.

```
git switch main lalu git pull --ff-only origin main
```

1.

```
git switch -c feat/<ringkasan>
```

1.

`git diff --check` dan `git status --short`

- 1.
- 2.
- 3.
- 4.
- 5.

Aturan commit

Gunakan gaya Conventional Commits:

- `feat`: add IT helpdesk intake form
- `fix`: persist task status after refresh
- `docs`: define branching workflow
- `refactor`: isolate LLM provider adapter
- `test`: cover helpdesk failure path
- `chore`: add environment example

Satu commit sebaiknya mewakili satu perubahan logis. Hindari pesan `update`, `changes`, atau `final`.

Aturan merge

- Jangan force-push branch bersama tanpa koordinasi.
- Jangan merge jika pemeriksaan yang relevan gagal, kecuali tim mencatat keputusan dan risikonya secara eksplisit.
- Hindari commit generated files, hasil build, cache, virtual environment, file rahasia, atau file upload pengguna.
- Konflik harus diselesaikan dengan memahami perubahan kedua sisi, bukan menghapus perubahan milik orang lain secara membabi buta.
- Untuk tenggat demo, lakukan freeze pada `main` setelah tim menyepakati kandidat final; semua perubahan sesudah freeze harus melalui review dan test cepat.

Pembagian kerja paralel

Sebelum membuat branch, sepakati pemilik area dan kontrak antarmuka agar anggota tidak mengubah file yang sama tanpa koordinasi. Contoh area: UI workspace, backend/workflow, document processing, Campus Twin, testing/deployment. Pembagian anggota spesifik: TBD sampai tim menentukan penanggung jawab.

Checklist sebelum PR/merge

- □ Scope perubahan sesuai tujuan branch.
- □ Tidak ada secret atau data pribadi.
- □ Test relevan ditambahkan/diperbarui dan dijalankan.

- □ Error/loading/empty states diperiksa untuk fitur UI.
- □ Dokumentasi diperbarui bila perilaku atau setup berubah.
- □ Mock/simulated data diberi label jelas.
- □ Diff sudah diperiksa dengan `git diff --check`.

Product Scope

Product Scope — DinusNexus

Product statement

DinusNexus adalah **Unified AI Digital Campus Worker**: satu platform human-centered yang menyatukan berbagai pekerjaan operasional kampus melalui conversational interaction, pemrosesan dokumen, workflow berbasis tools, bukti yang dapat diperiksa, serta pengawasan manusia.

Pengguna

Peran pengguna autentikasi dan matriks hak akses terperinci: TBD. Desain harus mendukung perbedaan hak akses dan tidak boleh menganggap semua orang dapat membaca semua data.

Delapan campus roles

Role	Pekerjaan yang didukung	Status implementasi baseline
Admissions Staff (PMB)	Menjawab pertanyaan calon mahasiswa dan memeriksa dokumen pendaftaran	Direncanakan; implementasi penuh TBD
Finance Staff	Menyiapkan pengingat dan merekonsiliasi catatan UKT/SPP	Direncanakan; implementasi penuh TBD
Academic Administration (BAAK)	Mendukung KRS, jadwal, dan surat keterangan	Direncanakan; implementasi penuh TBD
PDDikti Operator	Memvalidasi data mahasiswa/dosen sebelum pelaporan	Direncanakan; implementasi penuh TBD
IT Helpdesk	Mendukung reset akun dan triase Wi-Fi/jaringan	Workflow pertama yang diprioritaskan end-to-end
Quality Assurance Staff	Memetakan dokumen pendukung ke persyaratan akreditasi	Direncanakan; implementasi penuh TBD
Career Center Staff	Melaksanakan tracer study dan merangkum survei	Direncanakan; implementasi penuh TBD

Role	Pekerjaan yang didukung	Status implementasi baseline
Digital Archive Staff	Mengklasifikasi dan menemukan dokumen kampus	Direncanakan; implementasi penuh TBD

Shared product capabilities

- Satu workspace dan navigasi untuk semua role.
- Chat sebagai cara interaksi utama dan untuk input baru selama demo.
- Upload dokumen dan pemrosesan berdasarkan kebutuhan workflow.
- Stateful workflow agent: status tugas, langkah eksekusi, hasil, evidence, dan kegagalan tersimpan.
- Execution inspector yang menunjukkan status nyata, tools/sumber yang digunakan, serta hasil langkah.
- Hasil dan riwayat tetap tersedia setelah refresh.
- Persetujuan manusia sebelum tindakan sensitif.
- Audit log dan pengukuran penggunaan token AI.
- Campus Twin 2D interaktif sebagai konteks visual untuk lokasi/infrastruktur yang terkait.

Campus Twin

Campus Twin bukan role kesembilan. Ia adalah kemampuan lintas modul yang dapat menampilkan representasi gedung, zona, perangkat, dan insiden berdasarkan data yang tersedia. MVP memprioritaskan 2D interaktif. Bentuk data, jumlah gedung/perangkat, dan sumber integrasi final: TBD.

Batas prototype

Wajib diprioritaskan: satu workflow IT Helpdesk yang berjalan dari input hingga output, menggunakan setidaknya satu sumber data/tool selain LLM, menampilkan sumber/status/summary, menerima input baru saat demo, menangani kegagalan, dan menyimpan hasil/riwayat.

Tidak boleh diklaim tanpa implementasi dan bukti: integrasi sistem kampus live, diagnosis perangkat live, reset akun otomatis, pengiriman notifikasi, perubahan KRS, perubahan data PDDikti, transaksi pembayaran, atau semua role sudah fungsional.

Out of scope untuk MVP kecuali waktu memungkinkan

- Multi-agent system sebagai kebutuhan wajib.
- Campus Twin 3D penuh.
- Integrasi real ke semua sistem informasi kampus.
- Eksekusi perubahan sensitif tanpa persetujuan.
- Klaim bahwa semua delapan role memiliki workflow produksi lengkap.

Success criteria prototype

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

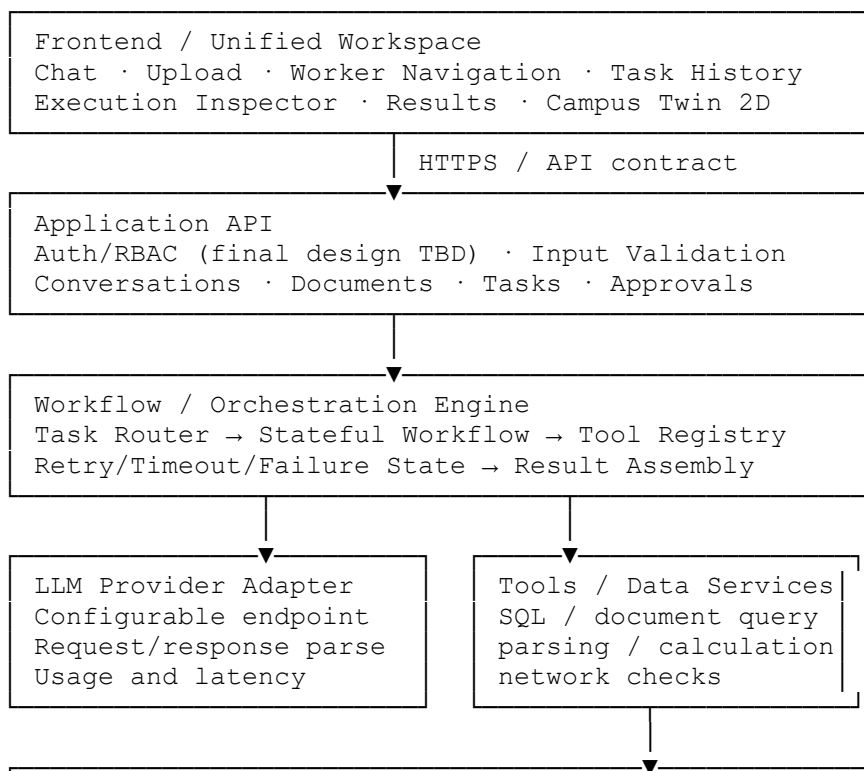
Architecture Baseline

Architecture Baseline

Prinsip

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

Logical architecture



Persistence Tasks/Runs/Steps · Evidence · Documents metadata Incidents/Devices · Audit · Token Usage
--

Component responsibilities

Frontend

- Kirim pesan, file, dan input baru.
- Menampilkan worker aktif, task status, execution timeline, sumber/evidence, output, dan error.
- Membaca riwayat dari backend agar bertahan setelah refresh.
- Tidak menyimpan API key AI di browser.

Application API

- Memvalidasi input dan file.
- Membuat dan membaca task/conversation.
- Mengirim pekerjaan ke workflow engine.
- Mengirim status dan hasil kembali ke frontend.
- Menegakkan otorisasi di server.

Workflow engine

- Menetapkan role/workflow.
- Membuat task run dan langkah eksekusi.
- Memanggil tools sesuai allowlist.
- Menyimpan outcome setiap langkah.
- Menangani timeout, kegagalan, kebutuhan approval, dan penyelesaian.

LLM provider adapter

- Menyembunyikan detail provider dari workflow.
- Memuat endpoint/model dari konfigurasi environment.
- Memformat request dan memvalidasi response.
- Mengambil usage metadata ketika provider mengembalikannya.
- Mengembalikan error yang aman tanpa mengekspos secret.

Provider, nama model, protokol API, dan schema autentikasi: TBD. Jangan mengasumsikan endpoint OpenAI-compatible sampai terverifikasi.

Tool registry

Mendaftarkan tools yang eksplisit dan diizinkan, misalnya lookup perangkat, pencarian dokumen, perhitungan, atau pemeriksaan jaringan yang aman. Tool harus mengembalikan data terstruktur beserta metadata sumber/waktu ketika tersedia.

Persistence

Menyimpan tugas, run, langkah, hasil, dan audit. Teknologi database dan file storage: TBD sampai repo dan budget diperiksa.

Campus Twin

Membaca building/zone/device/incident dari data yang tersedia dan memvisualisasikannya. UI tidak boleh mengubah status perangkat hanya melalui manipulasi tampilan. Sumber status dan label simulasi harus jelas.

Suggested domain entities

Entitas konseptual (bukan keputusan SQL final):

- User, Role, CampusWorker
- Conversation, Message
- Document, DocumentChunk jika retrieval chunking diperlukan
- Task, TaskRun, ExecutionStep
- Evidence, ToolCall
- Incident, CampusBuilding, CampusZone, CampusDevice, DeviceEvent
- Approval, AuditLog, TokenUsage

Implementasi boleh menggabungkan atau memisahkan tabel sesuai database dan pola repo.

Contract principle

Kontrak API perlu memakai ID stabil dan state eksplisit. Format final endpoint/DTO: TBD setelah audit stack. Minimal frontend dapat membuat task, membaca status/run steps, mengirim pesan/file, membuka hasil, dan membaca history. Status konseptual: `queued`, `running`, `waiting_for_approval`, `completed`, `failed`, `cancelled`.

Failure behavior

- Provider tidak tersedia: simpan error task dan tampilkan pesan aman.
- Tool gagal: tandai langkah gagal dan jangan menghasilkan klaim seolah tool sukses.
- Bukti kurang: nyatakan diagnosis belum pasti.
- Input tidak valid: tolak dengan pesan koreksi yang jelas.
- Upload gagal: tampilkan error dan jangan mengklaim dokumen telah diproses.
- Refresh/reconnect: muat status dari persistence.

Hal yang masih TBD

- Framework dan struktur repo aktual.
- Identitas/login dan model RBAC yang tepat.
- Model/provider/API panitia.
- Database dan strategi migrasi.
- Metode background execution/realtime update.
- Parser per jenis file.

- Sumber device/network data dan apakah real atau sintetis.
- Azure services dan domain/HTTPS setup.

Stateful Workflows

Stateful Workflows

Pola bersama

Semua campus worker harus dibangun menggunakan pola workflow yang bisa dilacak, walaupun isi langkahnya berbeda.

```

User Input / Document
    ↓
Validate & Identify Intent
    ↓
Create Task + Task Run
    ↓
Execute Allowed Steps / Tools
    ↓
Persist Step Status + Evidence
    ↓
Analyze / Calculate / Retrieve
    ↓
Assemble Result with Sources & Uncertainty
    ↓
Approval Gate if a consequential action is requested
    ↓
Persist Final Result and Audit Events
    ↓
Display Result and History
  
```

Pekerjaan boleh menghasilkan partial result jika sebagian tools gagal, tetapi UI harus menjelaskan bagian yang gagal dan batas kesimpulannya.

Task/run state

State machine konseptual:

- `queued`: diterima dan menunggu eksekusi.
- `running`: workflow sedang dieksekusi.
- `waiting_for_approval`: menunggu keputusan manusia.
- `completed`: langkah wajib berhasil dan hasil disimpan.
- `failed`: tidak dapat menghasilkan hasil yang valid.
- `cancelled`: dihentikan oleh pengguna/sistem.

Step status dapat lebih terperinci, termasuk `skipped` atau `retrying`, sesuai implementasi.

Workflow prioritas: IT Helpdesk

Input

- Deskripsi masalah.
- Lokasi/gedung atau zona, jika diketahui.
- Jenis perangkat atau layanan, jika diketahui.
- Waktu/indikasi kejadian, jika tersedia.
- Lampiran log/screenshot/dokumen yang didukung.

Steps

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.
- 11.
- 12.

Hasil

- Ringkasan insiden.
- Fakta/evidence yang ditemukan.
- Diagnosis atau status “belum cukup bukti”.
- Hipotesis dan bukti pendukung/penyangkal bila dievaluasi.
- Rekomendasi langkah berikutnya.
- Sumber/record ID.
- Status task/tiket.
- Waktu/durasi dan token usage apabila benar-benar terukur.

Data demo

Gunakan dataset sintetis yang jelas diberi label. Contoh gedung/perangkat tidak mewakili kondisi kampus nyata kecuali terintegrasi dengan sumber yang sah dan diverifikasi.

Workflow dokumen (shared capability)

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.

PDF scan/image mungkin memerlukan OCR; jika OCR tidak tersedia, nyatakan keterbatasan daripada berpura-pura file terbaca.

Worker-specific workflow outlines

- **PMB:** pertanyaan → retrieval sumber PMB → jawaban dengan referensi; upload → pemeriksaan kelengkapan → daftar item tersedia/kosong.
- **Finance:** data pembayaran → validasi schema → rekonsiliasi deterministic → exceptions → review manusia. Jangan membuat pembayaran atau mengirim pengingat tanpa izin.
- **BAAK:** permintaan → pencarian data/jadwal → verifikasi sumber → jawaban/draf. Perubahan catatan perlu otorisasi.
- **PDDikti:** dataset → validasi field/format/konsistensi → exception report → petugas meninjau. Jangan mengklaim pengiriman ke PDDikti tanpa integrasi.
- **IT Helpdesk:** intake → data/network evidence → analisis → rekomendasi → ticket/history.
- **Quality Assurance:** kriteria → cari dokumen bukti → mapping dengan sumber → gaps untuk review ahli.
- **Career Center:** data survei → pemeriksaan schema/anonymization → perhitungan → ringkasan berbasis hasil.
- **Digital Archive:** query → pencarian/classification → hasil berperingkat dengan metadata/link.

Outline ini adalah target desain. Hanya workflow yang telah diimplementasikan dan diuji boleh ditandai “implemented”.

Development Plan & Definition of Done

Development Plan & Definition of Done

Urutan prioritas

Pembagian kerja paralel untuk tiga anggota tim, ownership, checkpoint, dan tata kelola GitHub dijelaskan di [TEAM DEVELOPMENT PLAN.md](#). Dokumen ini tetap menjadi sumber urutan prioritas dan Definition of Done fitur.

P0 — Foundation and workflow

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.

10.

P1 — Campus Twin and evaluation

- 1.
- 2.
- 3.
- 4.
- 5.

P2 — Extend campus workers

After the IT Helpdesk workflow is stable, extend shared services to document search, PMB, Quality Assurance, or other role workflows based on time and data availability. Prioritization among these additional workers: TBD.

Audit-before-build checklist

- □ Inspect repository tree, branch, status, and recent commits.
- □ Identify frontend/backend framework and package manager.
- □ Identify database/storage already configured.
- □ Inspect existing UI and reusable components.
- □ Find tests, lint, type-check, build, and run commands.
- □ Inspect `.gitignore` and secret-handling patterns.
- □ Verify organizer API documentation and model availability; never infer from an API key alone.
- □ Record actual current state before changing architecture.

Definition of Done for a feature

A feature can be considered done only when:

- Its scope and UI/backend behavior are implemented, not only mocked.
- Inputs are validated and relevant failure cases are handled.
- Output and state persist where required.
- Source/evidence is displayed when the task relies on external data.
- Loading, empty, success, and error states are usable.
- Relevant tests or manual test steps are recorded with actual results.
- No secrets/personal data are accidentally committed.
- Documentation is updated if setup, API, workflow, or product behavior changes.
- Any mock/simulated component is labeled in the application and README.

Demo readiness checklist

- One complete IT Helpdesk scenario works end-to-end.
- A fresh input can be submitted live.
- At least one source/tool beyond the LLM is exercised.
- User can see steps/status and supporting evidence.
- Final result and history survive refresh.
- One failure case is handled and presented clearly.
- Token usage is measured or marked unavailable if provider does not expose it.
- New/synthetic data is disclosed.
- Video backup is recorded.
- Final commit is recorded before submission cutoff.

Rencana Development Paralel Tim

Rencana Development Paralel Tim — DinusNexus

Tujuan dokumen

Dokumen ini menerjemahkan baseline produk DinusNexus menjadi rencana kerja paralel untuk **tiga orang anggota tim**. Rencana ini melengkapi [DEVELOPMENT PLAN.md](#), bukan menggantikan keputusan arsitektur, scope, keamanan, atau workflow yang sudah tercatat di dokumen lain.

Status dokumen: **PROPOSED** Tanggal penyusunan: 2026-10-09 Target utama: demo prototype IT Helpdesk end-to-end yang dapat diuji dari input baru sampai hasil dan history.

Target bersama

Pada akhir timebox, tim menargetkan satu alur berikut berjalan dan dapat dibuktikan:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.

Fitur role lain dan Campus Twin 2D dikerjakan setelah jalur IT Helpdesk stabil atau hanya jika tidak mengganggu target demo.

Pembagian role tiga orang

Nama anggota bersifat TBD; gunakan nama GitHub aktual setelah tim menyepakatinya.

Role	Tanggung jawab utama	Deliverable utama	Tidak boleh menjadi bottleneck
Anggota A — Product & Frontend Lead	Unified Workspace, chat/input baru, upload UX, execution inspector, result/history, state UI, accessibility	UI yang memanggil kontrak API, menampilkan state nyata, error/loading/empty state, dan history setelah refresh	Menggunakan mock lokal sebagai pengganti kontrak backend setelah API skeleton tersedia
Anggota B — Backend & Workflow Lead	API, task/run/step state machine, IT Helpdesk orchestration, tool registry, evidence, failure/approval state	Endpoint tervalidasi, workflow Helpdesk end-to-end, persistence, tool data sintetis berlabel, error aman	Menunggu UI selesai; kontrak API dipublikasikan lebih awal dengan fixture
Anggota C — Data, Quality & Delivery Lead	Dataset/tool adapter, LLM provider adapter, test strategy, security checks, token metrics, Azure/CI readiness	Fixture dan tool terstruktur, provider adapter configurable, test case, CI checks, smoke test, deployment/fallback plan	Mengambil keputusan stack/provider yang masih TBD tanpa verifikasi atau approval

Tanggung jawab bersama

- Semua anggota membaca [AGENTS.md](#), [CONTRIBUTING.md](#), dan dokumen scope terkait sebelum coding.
- Tidak ada secret, data pribadi nyata, atau credential cloud di repository.
- Perubahan lintas area harus dibahas sebelum dikerjakan; hindari mengubah file yang sama secara bersamaan.
- Setiap anggota wajib menulis hasil test aktual pada PR dan memperbarui dokumentasi jika kontrak/perilaku berubah.
- Review silang minimal satu orang sebelum merge; untuk perubahan keamanan, persistence, approval, atau deployment, minta review Anggota C.

Kontrak antarmuka sebelum implementasi paralel

Backend harus menerbitkan kontrak awal paling lambat setelah fase audit. Kontrak boleh berupa DTO/schema sesuai stack aktual, tetapi minimal memuat:

```
CreateTaskInput
  worker: "it_helpdesk"
  description: string
  location?: string
  device_type?: string
  attachments?: file_id[]
```

```

TaskSummary
  task_id: string
  run_id: string
  worker: string
  status: queued | running | waiting_for_approval | completed | failed |
cancelled
  created_at: timestamp

ExecutionStep
  step_id: string
  name: string
  status: queued | running | completed | failed | skipped | retrying
  source_ids?: string[]
  error_code?: string
  started_at?: timestamp
  completed_at?: timestamp

TaskResult
  facts: item[]
  interpretation: item[]
  uncertainty: item[]
  recommendations: item[]
  evidence: Evidence[]
  approval?: ApprovalRequest

```

Detail field, endpoint, transport realtime, dan database tetap **TBD** sampai audit repository selesai. Perubahan kontrak harus melalui PR yang menyertakan dampak ke frontend, backend, fixture, dan test.

Fase kerja paralel

Urutan ini memakai dependency minimum sehingga setiap anggota dapat mulai dengan fixture/kontrak, bukan menunggu implementasi penuh anggota lain.

Fase 0 — Kickoff dan audit bersama

Output gate: semua anggota memahami scope, stack aktual, branch strategy, dan daftar TBD.

- A: audit UI, routing, komponen reusable, command build/test.
- B: audit API/backend, persistence, konfigurasi environment, task model.
- C: audit package/lock, test/lint/CI, secret handling, Azure dan organizer API.
- Bersama: catat hasil audit pada issue GitHub; jangan mengubah stack sebelum diverifikasi.

Fase 1 — Foundation paralel

Anggota A

- Buat shell Unified Workspace dan navigasi minimal.
- Buat state UI untuk idle, submitting, running, success, partial, approval, failed.
- Integrasikan fixture `TaskSummary`, `ExecutionStep`, dan `TaskResult`.

Anggota B

- Implementasikan task/run/step persistence dan state transition.
- Sediakan endpoint create task, get task/run, get history.
- Implementasikan workflow skeleton dengan step yang dapat dilacak.

Anggota C

- Siapkan dataset sintetis gedung/zona/perangkat/insiden.
- Implementasikan tool adapter read-only dengan source ID, timestamp, dan label simulasi.
- Siapkan provider adapter configurable; jika detail provider belum tersedia, gunakan fake provider untuk test tanpa mengklaim integrasi nyata.
- Tambahkan test fixture untuk input valid, input kosong, tool failure, dan provider failure.

Output gate: UI dapat menampilkan fixture dan API dapat mengembalikan state minimal tanpa secret.

Fase 2 — Integrasi IT Helpdesk

Anggota A

- Hubungkan form/chat ke create-task endpoint.
- Tampilkan task ID, timeline step, evidence, result, dan retry/error state.
- Pastikan refresh memuat ulang data dari backend, bukan state memory saja.

Anggota B

- Hubungkan workflow ke tool perangkat/lokasi dan histori insiden.
- Simpan evidence tiap tool dan hasil partial bila sebagian step gagal.
- Tambahkan approval gate untuk tindakan berdampak; default prototype adalah rekomendasi, bukan eksekusi otomatis.

Anggota C

- Validasi schema dan sanitasi input/file di sisi server bersama Anggota B.
- Tambahkan provider usage/latency/token capture jika metadata benar-benar tersedia; bila tidak, tampilkan `unavailable`.
- Tambahkan integration test dan verifikasi tidak ada secret/data pribadi pada log.

Output gate: satu laporan baru mengalir dari UI → API → workflow/tool → evidence/result → history.

Fase 3 — Hardening, Campus Twin, dan delivery

Anggota A

- Tambahkan tampilan Campus Twin 2D minimal hanya dari dataset aktif.
- Tampilkan badge simulasi dan tautkan device/location ke incident jika relasinya valid.

- Periksa keyboard focus, label form, kontras, dan responsive layout.

Anggota B

- Perbaiki timeout, retry terbatas, cancellation, dan reconnect/polling sesuai kemampuan stack.
- Pastikan status `failed` tidak menghasilkan hasil seolah-olah sukses.
- Tambahkan audit event untuk task, tool, approval, dan error.

Anggota C

- Konfigurasi GitHub Actions untuk lint, type-check, unit/integration test, dan build.
- Siapkan smoke test deployment serta estimasi Azure; jangan membuat resource berbayar tanpa persetujuan.
- Siapkan fallback localhost/video demo dan checklist teardown.

Output gate: demo rehearsal lulus dengan input baru, satu failure case, refresh history, evidence, dan label data simulasi.

Strategi GitHub

Repository dan proteksi branch

- `main` adalah branch stabil; aktifkan branch protection dan larang direct push.
- Wajib status check CI sebelum merge.
- Aktifkan CODEOWNERS jika repository mengizinkan.
- Gunakan GitHub Issues untuk pekerjaan terukur, GitHub Projects untuk board/status, Pull Request untuk review, dan GitHub Actions untuk pemeriksaan otomatis.

Branch per pekerjaan

Branch dibuat dari `main` terbaru dan hanya memuat satu tujuan:

```
feat/workspace-shell
feat/helpdesk-state-machine
feat/synthetic-helpdesk-tool
test/helpdesk-failure-path
chore/ci-checks
docs/team-development-plan
```

Jangan memakai branch bersama seperti `anggota-a` atau `update`. `Rebase/merge` dari `main` dilakukan sebelum PR bila branch tertinggal jauh; jangan `force-push` branch yang sedang dipakai orang lain.

Issue dan Project board

Buat satu GitHub Issue untuk setiap deliverable yang dapat selesai. Gunakan label:

- `area:frontend`, `area:backend`, `area:data`, `area:qa`, `area:docs`, `area:deployment`

- `priority:p0, priority:p1, priority:p2`
- `status:blocked, status:needs-decision`
- `type:feature, type:bug, type:test, type:chore`

Kolom Project yang disarankan:

Backlog → Ready → In Progress → Review → Blocked → Done

Issue minimal memiliki tujuan, acceptance criteria, owner, dependency, dan cara verifikasi. Jika ada keputusan TBD, buat issue `status:needs-decision` dan tautkan ke `DECISIONS.md`.

Pull Request

Judul PR mengikuti Conventional Commits, misalnya `feat: persist helpdesk task steps`.

Template PR minimal:

- Ringkasan dan issue terkait (`Closes #...` bila tepat).
- Scope dan file utama.
- Kontrak API/schema yang berubah.
- Cara test dan output aktual.
- Screenshot/video untuk perubahan UI.
- Dampak keamanan, secret, data simulasi, dan approval.
- Dokumentasi yang diperbarui.
- Risiko dan langkah rollback/fallback.

Aturan merge:

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

Jadwal dan checkpoint

Gunakan durasi relatif agar tetap berlaku terhadap deadline hackathon yang belum tercantum.

Checkpoint	Target	Bukti yang harus ada
C0 — selesai kickoff	Audit dan pembagian ownership selesai	GitHub Issues, Project board, daftar TBD
C1 — foundation	Kontrak dan skeleton UI/API tersedia	PR A/B/C, fixture, CI dasar
C2 — vertical slice	Happy path Helpdesk berjalan	PR integrasi, task/run/step tersimpan
C3 — hardening	Failure path, evidence, refresh history, security check	Test report dan screenshot

Checkpoint	Target	Bukti yang harus ada
C4 — freeze	Kandidat demo tidak berubah tanpa review	Tag/release candidate, video backup
C5 — submission	Demo dan dokumentasi final	Commit final, deployment/fallback checklist

Jika waktu menyempit, hentikan pekerjaan P1/P2 terlebih dahulu. Jangan mengorbankan happy path IT Helpdesk, evidence, persistence, failure state, atau secret safety demi menambah role.

Risiko dan mitigasi

Risiko	Dampak	Mitigasi	Owner
Stack atau provider API belum terverifikasi	Rework dan integrasi gagal	Audit lebih dulu; gunakan adapter dan fake provider pada test	B + C
Perubahan kontrak memblokir frontend	Integrasi terlambat	Publikasikan schema awal dan fixture; perubahan lewat PR	B
Tool/data demo dianggap live	Klaim produk menyesatkan	Badge <code>SIMULATED DATA</code> , source metadata, disclosure README	C
Workflow menghasilkan diagnosis tanpa bukti	Keputusan tidak dapat dipercaya	Evidence wajib, uncertainty eksplisit, partial/failure state	B
Secret masuk log atau client bundle	Insiden keamanan	Environment variable, redaction, CI secret scan, review C	C
Azure menambah biaya atau risiko demo	Deployment gagal/biaya tak terduga	Localhost fallback, estimasi biaya, approval resource, teardown plan	C
Konflik file antaranggota	Integrasi lambat	Ownership per area, PR kecil, koordinasi sebelum file lintas area diubah	Semua

Definition of Done tim

Pekerjaan dinyatakan selesai hanya jika:

- acceptance criteria Issue terpenuhi;
- implementasi dan test relevan tersedia, bukan hanya mock yang tidak diberi label;
- input, loading, empty, success, partial, approval, dan failure state ditangani sesuai konteks;
- output yang bergantung pada tool memiliki evidence/source;
- state penting bertahan setelah refresh;
- tidak ada secret atau data pribadi nyata pada diff, log, fixture, atau screenshot;
- CI dan pemeriksaan lokal yang relevan lulus;
- dokumentasi terkait diperbarui;
- PR telah direview dan merge ke `main`.

Pembagian ownership setelah merge

Ownership bukan berarti hanya satu orang yang boleh memperbaiki area tersebut. Pemilik bertugas menjaga konsistensi dan menjadi reviewer utama:

Area	Primary Backup	
Workspace, chat, inspector, Campus Twin UI	A	C
API, state machine, persistence, approval	B	A
Dataset, tool adapter, provider, CI, security, deployment	C	B
README, decision register, demo disclosure	C	A

Perubahan kebijakan produk atau keputusan yang berstatus TBD tetap membutuhkan persetujuan tim dan pembaruan [DECISIONS.md](#), bukan keputusan sepihak pemilik area.

UI/UX Rules

UI/UX Rules

Pengalaman produk

DinusNexus harus terasa seperti rekan kerja digital: mudah diajak bicara, menerima dokumen dan tugas, memperlihatkan proses kerjanya, serta meminta bantuan/approval manusia ketika diperlukan.

Layout utama

Gunakan satu Unified Workspace dengan tiga area konseptual:

- 1.
- 2.
- 3.

Implementasi responsif boleh menggabungkan panel pada layar kecil, tetapi kemampuan inti tidak boleh hilang.

Chat dan input

- Pengguna dapat memulai tugas baru dengan bahasa alami.
- Pengguna dapat memilih campus worker atau membiarkan router mengklasifikasikan permintaan jika fitur itu sudah diimplementasikan.
- Tersedia input teks dan kontrol upload dokumen yang jelas.
- Tampilkan nama file, ukuran jika tersedia, status upload, status parsing, serta kegagalan.
- Mendukung instruksi lanjutan dan input baru selama demo.
- Jangan menampilkan tombol yang seolah bekerja tetapi belum terhubung; tandai unavailable/coming later atau sembunyikan dari MVP.

Execution Inspector

Setiap step harus menampilkan status aktual dari backend: queued/running/completed/failed/skipped/awaiting approval, sesuai state machine implementasi.

Informasi yang berguna:

- Task ID dan worker.
- Ringkasan langkah yang dijalankan.
- Tool/sumber data yang diakses.
- Hasil atau ringkasan evidence.
- Durasi jika diukur.
- Error dan tindakan pemulihan jika gagal.
- Token usage jika provider menyediakannya.

Jangan tampilkan chain-of-thought privat. Tampilkan execution trace, bukti, hasil tool, dan alasan operasional yang dapat diperiksa.

Result design

Pisahkan secara visual:

- **Temuan/fakta:** berasal dari sumber atau hasil tool.
- **Interpretasi AI:** analisis berdasarkan bukti yang terlihat.
- **Ketidakpastian:** hal yang belum bisa dipastikan.
- **Rekomendasi:** langkah berikutnya.
- **Approval:** tindakan yang menunggu persetujuan jika ada.

Setiap sumber harus punya label yang cukup untuk ditelusuri, seperti nama dokumen, ID record, waktu pengambilan, atau ID tool result jika tersedia.

Visual direction

- Tampilan produk operasional yang profesional dan bersih, bukan halaman promosi.
- Hierarki informasi jelas; jangan membuat dashboard terlalu padat dengan kartu statistik dekoratif.
- Gunakan komponen UI konsisten dan aksesibel.
- Gunakan warna status secara konsisten: berjalan, sukses, perhatian, gagal, dan menunggu approval.
- Gunakan skeleton/loading state hanya saat request benar-benar sedang berjalan.
- Pastikan kontras, keyboard focus, label form, dan pesan validasi dapat digunakan.
- Stack dan design tokens spesifik: TBD setelah audit repository. Jangan memasang design system baru jika repo sudah memiliki sistem yang sesuai.

Campus Twin 2D

- Menampilkan objek yang benar-benar ada di dataset aktif.

- Gunakan legenda dan status yang mudah dipahami.
- Saat memilih gedung/zona/perangkat, tampilkan detail dan insiden terkait.
- Klik insiden pada task harus dapat membuka konteks device/location jika relasinya tersedia.
- Data sintetis atau simulasi harus terlihat jelas diberi badge/label.
- Versi 3D bukan syarat MVP.

State wajib di UI

Minimal setiap workflow mendukung: idle/empty, submitting, running, success, partial result, waiting for approval, failed, dan retry/recovery jika didukung.

Live-demo checklist

- `<span class="box">#9633;</span>` Masukkan laporan baru yang belum disiapkan sebelumnya.
- `<span class="box">#9633;</span>` Unggah file yang valid dan lihat status pemrosesan.
- `<span class="box">#9633;</span>` Lihat execution steps yang berubah dari backend.
- `<span class="box">#9633;</span>` Buka bukti/sumber hasil.
- `<span class="box">#9633;</span>` Tunjukkan pesan error pada satu input atau tool yang gagal dengan aman.
- `<span class="box">#9633;</span>` Refresh halaman dan buktikan history tetap ada.
- `<span class="box">#9633;</span>` Tunjukkan label data simulasi bila ada.

Data, Security & Human Oversight

Data, Security & Human Oversight

Prinsip dasar

- Gunakan data sintetis atau anonim untuk demo dan pengembangan.
- Data kampus hanya boleh digunakan jika diizinkan dan tidak bersifat rahasia tanpa otorisasi yang tepat.
- API key, token, password, connection string, dan credential cloud tidak boleh disimpan di repository.
- Terapkan akses minimum yang diperlukan.
- Semua tindakan penting harus dapat ditelusuri.

Secrets

- Simpan rahasia pada environment variables atau secret store yang sesuai.
- Commit `.env.example` dengan nama variabel dan placeholder, bukan nilai asli.
- Pastikan `.env`, credential, file service principal, dan dump berisi data pribadi masuk `.gitignore`.

- Jangan log request header autentikasi atau API key.
- Jika secret terlanjur di-commit, cabut/rotate secret terlebih dahulu; menghapus dari commit terbaru saja tidak cukup.

Dokumen upload

- Validasi file di server: jenis yang didukung, ukuran, nama file, dan hasil parsing.
- Jangan percaya MIME type dari browser saja.
- Hindari menggunakan nama file sebagai path penyimpanan.
- Berikan ID file internal dan simpan metadata dengan akses terkontrol.
- Jangan memproses file dengan parser yang tidak sesuai.
- Jangan mengirim seluruh dokumen kepada provider AI jika subset relevan cukup.
- File dan teks di dalamnya dapat berisi prompt berbahaya. Perlakukan konten dokumen sebagai data tidak tepercaya, bukan instruksi yang boleh mengubah policy, membuka secret, atau menjalankan tool sewenang-wenang.
- Retensi dan penghapusan file: TBD; tetapkan sebelum menggunakan dokumen sensitif nyata.

AI and tools

- LLM tidak dianggap sebagai otoritas sumber data operasional.
- Fakta jaringan, pembayaran, jadwal, atau status dokumen harus berasal dari data/tool.
- Gunakan allowlist tool dan validasi argumen.
- Jangan memberikan akses shell/database/cloud tanpa batas kepada model.
- Tool yang mengubah keadaan harus memiliki policy check dan, bila sensitif, approval manusia.
- Data dari model divalidasi sebelum disimpan atau digunakan untuk tindakan berikutnya.
- Tangani prompt injection dari dokumen dan hasil tool sebagai risiko.

Human approval

Brief hackathon mewajibkan persetujuan staf sebelum mengirim pesan, mengubah catatan akademik, atau menyetujui pembayaran. DinusNexus menerapkan prinsip serupa untuk tindakan berdampak.

Contoh tindakan yang harus dilindungi: reset akun, perubahan konfigurasi jaringan, pengiriman komunikasi eksternal, perubahan data akademik, pembayaran, atau pengubahan sumber data penting. Tindakan spesifik dan role yang berwenang: TBD berdasarkan kebijakan nyata.

Sebelum tindakan sensitif:

- 1.
- 2.
- 3.
- 4.
- 5.

Audit

Audit event sebaiknya mencakup task ID, actor/user ID jika ada, worker/tool, aksi, status, timestamp, sumber terkait, approval, serta correlation ID. Hindari mencatat seluruh isi dokumen atau rahasia jika tidak dibutuhkan.

Simulated data

Gunakan label seperti `SIMULATED DATA` atau `DEMO DATA` pada UI, README, dan demo. Dokumentasikan skenario, field simulasi, dan bagian yang belum terhubung ke sistem live.

AI Token Efficiency & Measurement

AI Token Efficiency & Measurement

Tujuan

Mengukur penggunaan token dan mengoptimalkan biaya/pemakaian tanpa menurunkan akurasi. Efisiensi token merupakan salah satu kriteria penilaian resmi; karena itu, klaim optimasi harus disertai pengukuran.

Logging yang disarankan

Untuk setiap pemanggilan provider, simpan jika metadata tersedia:

- task/run ID dan step ID.
- provider/model identifier yang tidak mengandung secret.
- timestamp dan latency.
- input token dan output token dari usage metadata provider.
- status sukses/gagal.
- jumlah retry.
- kategori tugas/worker.
- estimasi biaya jika rate dan aturan kalkulasinya sudah diverifikasi.

Jangan mengarang token count. Jika provider tidak mengembalikan usage, tandai nilai sebagai `unavailable` atau gunakan estimator dan jelaskan bahwa itu estimasi.

Optimasi prioritas

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.

- 7.
- 8.
- 9.

Baseline experiment

Bandungkan baseline dan optimized approach dengan input serta dataset yang sama.

Catat:

- akurasi/tingkat keberhasilan task.
- input/output token aktual atau estimasi yang ditandai.
- latency.
- jumlah pemanggilan model/tool.
- error/failure rate.
- kelengkapan source/evidence.

Template hasil:

Test case	Approach	Success/accuracy	Input tokens	Output tokens	Latency	Notes
TBD	Baseline	TBD	TBD	TBD	TBD	Belum diukur
TBD	Optimized	TBD	TBD	TBD	TBD	Belum diukur

Jangan menyimpulkan penghematan atau mempertahankan akurasi sebelum pengujian dilakukan.

Dashboard token

Tampilkan agregat dan penggunaan per task bila metadata tersedia. Jelaskan apakah angka berasal dari provider atau estimator. Jangan menampilkan API key, prompt sensitif, atau isi dokumen privat di dashboard usage.

Azure Deployment Baseline

Azure Deployment Baseline

Yang sudah diputuskan

- Target deployment adalah Microsoft Azure.
- Anggaran kredit yang tersedia diperkirakan sekitar **US\$100**.
- Pengeluaran infrastruktur perlu dikendalikan dan diawasi.
- Localhost tetap dapat digunakan untuk demo jika deployment cloud menambah risiko atau mengganggu kesiapan.

- Kredit untuk infrastruktur berbeda dari alokasi token AI panitia.

Yang belum diputuskan

- Azure subscription/resource group dan region.
- Layanan hosting frontend/backend.
- Database layanan atau database lokal untuk demo.
- File/object storage.
- Application Insights/monitoring.
- Domain, TLS, networking, dan identitas deployment.
- Jumlah instance/ukuran resource dan estimasi biaya akhir.

Semua itu tetap TBD sampai diperiksa pada subscription yang benar. Jangan membuat resource berbayar tanpa persetujuan pemilik subscription.

Strategi pengendalian biaya

- 1.
- 2.
- 3.
- 4.
- 5.
- 6.
- 7.
- 8.
- 9.
- 10.

Deployment checklist

- □ Repo bersih dan commit kandidat demo diketahui.
- □ Test lokal selesai dan hasilnya dicatat.
- □ Environment variables disiapkan di luar source code.
- □ API key provider dan secret database tidak masuk build/client.
- □ Azure resource/cost estimate sudah ditinjau.
- □ Seed/demo data diberi label.
- □ Smoke test: login/access, chat, upload, run task, result, history refresh.
- □ Error provider/tool diuji.
- □ Token usage/logging diverifikasi.
- □ Teardown/cleanup plan dicatat.

Catatan

Pilihan layanan Azure tertentu tidak diputuskan dalam dokumen ini karena membutuhkan pemeriksaan stack aktual, subscription, region, dan harga. Jangan menganggap kredit US\$100 pasti mencukupi setiap arsitektur; gunakan opsi termurah yang memenuhi kebutuhan demo setelah estimasi biaya diverifikasi.

Decision Register

Decision Register

Dokumen ini membedakan hal yang sudah diputuskan dari hal yang masih perlu dikonfirmasi. Jangan mengubah item TBD menjadi keputusan final tanpa konfirmasi tim.

DECIDED — sudah disepakati

ID	Keputusan	Detail
D-001	Nama produk	DinusNexus
D-002	Positioning	Unified AI Digital Campus Worker untuk operasional kampus
D-003	Campus roles	Satu platform mencakup delapan role dari brief CBN: PMB, Finance, BAAK, PDDikti, IT Helpdesk, Quality Assurance, Career Center, Digital Archive
D-004	Implementasi awal	IT Helpdesk menjadi workflow pertama yang diprioritaskan end-to-end
D-005	Platform breadth	Satu workspace/navigasi untuk seluruh role; tidak wajib semua role langsung memiliki workflow lengkap
D-006	AI interaction	Pengguna dapat chat dan memberikan input baru selama demo
D-007	Documents	Pengguna dapat upload dokumen; format yang didukung ditetapkan setelah stack/parser dipastikan
D-008	Transparent execution	UI perlu memperlihatkan status dan proses kerja yang berasal dari backend, plus hasil dan sumber/evidence yang relevan
D-009	Persistence	Hasil/riwayat harus tetap tersedia setelah refresh
D-010	Workflow pattern	Stateful workflow agent (D3)
D-011	Campus Twin	Campus Twin sebagai kemampuan tambahan dan konteks visual; mulai dari 2D interaktif (E4), bukan mewajibkan 3D
D-012	Human oversight	Tindakan sensitif memerlukan kontrol/approval manusia sesuai kewenangan dan aturan
D-013	Coding workflow	OpenCode digunakan sebagai coding agent; API key disediakan oleh panitia
D-014	Deployment target	Azure, dengan kredit yang diperkirakan sekitar US\$100; layanan spesifik belum diputuskan

ID	Keputusan	Detail
D-015	Documentation	Gunakan dokumen Markdown terpisah dengan status keputusan yang eksplisit

PROPOSED — rekomendasi, belum final

ID	Rekomendasi	Mengapa belum final
P-001	Next.js + TypeScript + Tailwind + shadcn/ui untuk UI	Stack repository aktual belum diaudit dan pengguna belum memilih framework
P-002	FastAPI/Python untuk backend	Stack repository dan kebutuhan deployment belum dikonfirmasi
P-003	PostgreSQL untuk data persisten	Database saat ini dan biaya Azure belum dikonfirmasi
P-004	Azure Blob Storage untuk dokumen pada deployment	Keputusan bergantung pada kebutuhan, biaya, dan integrasi aktual
P-005	Satu orchestration engine dengan workflow khusus per role, bukan multi-agent	Sesuai timebox; implementasi akhir menyesuaikan kebutuhan nyata

TBD — belum diputuskan/harus diverifikasi

Area	Pertanyaan yang belum dijawab
Repository	Struktur kode, framework, branch, perubahan lokal, dan komponen yang sudah tersedia
Organizer API	Provider/model, base URL, endpoint, autentikasi, payload, token usage metadata, rate limit, dan dokumentasi resmi
Frontend/backend	Framework dan versi aktual yang harus dipakai
Database	Mesin database, schema final, migrasi, dan lokasi penyimpanan
Authentication	Login, SSO bila ada, role-based access policy, serta akun demo
File upload	Format final, batas ukuran, parser, OCR bila diperlukan, retensi, dan mekanisme penghapusan
Tools Helpdesk	Dataset perangkat, sumber status jaringan, pemeriksaan yang aman, dan mana yang disimulasikan
Campus Twin	Data gedung/zona/perangkat, sumber, koordinat/layout, dan cakupan awal
Background/realtime	Polling, Server-Sent Events, WebSockets, atau mekanisme lain sesuai stack
Azure	Subscription/resource group, region, layanan hosting, database, storage, budget alert, domain, dan estimasi biaya
Team	Penanggung jawab setiap area, review owner, dan jadwal freeze
Additional workers	Urutan role tambahan setelah workflow IT Helpdesk stabil

Change control

Jika ada keputusan baru:

- 1.
- 2.
- 3.
- 4.

Markdown Documentation Policy

Markdown Documentation Policy

Tujuan

Dokumentasi harus memberi satu sumber kebenaran yang konsisten bagi anggota tim, OpenCode, dan reviewer. Dokumen yang sudah pasti ditetapkan sebagai aturan; detail belum pasti ditandai, bukan ditebak.

Bahasa dan gaya

- Bahasa utama dokumentasi internal: Bahasa Indonesia; istilah teknis umum boleh menggunakan bahasa Inggris.
- Judul file dan heading harus jelas, singkat, dan stabil.
- Satu dokumen memiliki satu tujuan utama.
- Jelaskan keputusan, alasan, batasan, dan tindakan yang diperlukan.
- Gunakan tabel untuk status/opsi dan checklist untuk tindakan operasional.
- Jangan membuat klaim fitur, integrasi, hasil test, penggunaan token, atau deployment jika belum dibuktikan.

Struktur file

- `README.md`: overview produk, scope, cara mulai, dan indeks dokumen.
- `AGENTS.md`: instruksi kerja coding agent.
- `CONTRIBUTING.md`: branching, commit, PR, review.
- `docs/PRODUCT_SCOPE.md`: visi, pengguna, delapan role, scope MVP.
- `docs/ARCHITECTURE.md`: komponen, tanggung jawab, kontrak, kegagalan.
- `docs/UI_UX.md`: aturan interaksi dan design direction.
- `docs/WORKFLOWS.md`: state machine dan alur per role.
- `docs/DATA_SECURITY.md`: data, secret, upload, approval, audit.
- `docs/TOKEN_EFFICIENCY.md`: logging dan evaluasi token.
- `docs/AZURE_DEPLOYMENT.md`: konfigurasi deployment dan biaya.
- `docs/DEVELOPMENT_PLAN.md`: prioritas dan Definition of Done.
- `docs/DECISIONS.md`: daftar keputusan final dan pertanyaan TBD.

Tambahkan dokumen lain hanya jika ada kebutuhan jelas, misalnya API contract atau test plan yang sudah mulai konkret.

Aturan status keputusan

Gunakan:

- **DECIDED**: disepakati eksplisit oleh pengguna/tim.
- **PROPOSED**: rekomendasi yang belum disetujui.
- **TBD**: belum diketahui atau memerlukan konfirmasi.
- **IMPLEMENTED**: dikonfirmasi oleh kode dan pengujian yang relevan.
- **VERIFIED**: diuji dalam kondisi yang disebutkan, dengan bukti/hasil tercatat.

Jangan mengubah `PROPOSED` atau `TBD` menjadi `DECIDED` hanya karena coding agent menyarakannya.

Update policy

- Perubahan fitur/arsitektur harus memperbarui dokumen terkait.
- Keputusan yang berdampak lintas modul dicatat di `docs/DECISIONS.md`.
- Catat tanggal serta alasan keputusan penting jika diketahui.
- Jangan duplikasi rincian besar di banyak file; buat tautan ke dokumen sumber.
- Dokumentasikan command setup/test aktual dari repo dan pastikan command tersebut sudah diperiksa.
- Daftar TBD harus dipertahankan sampai ada keputusan eksplisit.

Markdown quality checklist

- `<span class="box">#9633;</span>` Heading mengikuti hierarki logis.
- `<span class="box">#9633;</span>` Link internal menuju path yang benar.
- `<span class="box">#9633;</span>` Code blocks memiliki language tag jika sesuai.
- `<span class="box">#9633;</span>` Tidak ada secret, data pribadi, atau credential.
- `<span class="box">#9633;</span>` Tidak ada klaim testing/deployment tanpa hasil nyata.
- `<span class="box">#9633;</span>` Status `DECIDED/PROPOSED/TBD/IMPLEMENTED/VERIFIED` digunakan secara konsisten.